

1 · Anforderungen

Webanwendung mit Next.js App Router für Spielrunden: Registrierung, Anmeldung, geschützte Spiel- und Ranglistenseiten, Logout. Alle Daten werden zur Laufzeit im Speicher eines einzelnen Node-Prozesses gehalten.

Anforderung	Akzeptanzkriterium	Stand
Registrierung	Anzeigename, E-Mail, Passwort; eindeutige normalisierte E-Mail; ungültige Eingaben sicher abgelehnt.	Verifiziert
Anmeldung	Serverseitig prüfbare Sitzung; ungültige Zugangsdaten neutral abgelehnt.	Verifiziert
Virtuelle Credits	Startguthaben 100, genau einmal vergeben; sichtbar im geschützten Bereich; konsistent bei Spielrunden verändert.	Verifiziert
Spielseite(n)	Dice und Roulette: vollständige, serverseitig entschiedene, atomar abgerechnete Runde.	Verifiziert
Leaderboard	Rangliste nach Creditstand, keine privaten Zugangsdaten.	Verifiziert
Laufzeitdaten	Konto-, Sitzungs-, Credit- und Spieldaten bleiben im selben Serverprozess erhalten.	Verifiziert
Navigation	Alle Hauptbereiche über echte Links/Buttons erreichbar, nicht nur per URL.	Verifiziert
Logout	Sitzung wird serverseitig ungültig; Konto/Credits/Spieldaten bleiben für erneuten Login erhalten.	Verifiziert

Bewusst nicht umgesetzt: persönliche Spielhistorie (`/history`), Reveal-Animationen, lokales Audio — nachrangig, ohne Einfluss auf den Pflichtumfang. **Out of Scope:** OAuth, E-Mail-Verifikation, Passwort-Reset, MFA, Echtgeld, reale Datenbank, WebSockets, Multiplayer, drittes Spiel, Adminbereich.

Registrierung & Startguthaben (Kernregeln)

- E-Mail getrimmt, kleingeschrieben, eindeutig; Passwort 12–128 Zeichen, Argon2id-Hash, nie im Klartext gespeichert.
- Client liefert nur Anzeigename/E-Mail/Passwort — Konto-ID, Rolle, Guthaben, Transaktionen sind serverseitig bestimmt.
- Startguthaben 100 Credits, genau eine Transaktion mit Grund `STARTING_CREDIT`; Konto+Transaktion werden atomar (vollständig oder gar nicht) geschrieben.

Anmeldung, Sitzung, Logout

- Argon2id-Passwortvergleich inkl. Dummy-Hash bei unbekannter E-Mail (Timing-Schutz); falsche/unbekannte/ungültige Zugangsdaten liefern dieselbe neutrale Meldung.
- Kryptografisch zufälliges, undurchsichtiges Sitzungstoken, 8 h Ablauf; im Store nur SHA-256-Hash. Cookie: `HttpOnly`, `SameSite=Lax`, in Produktion `Secure`.
- Same-Origin/`Sec-Fetch-Site`-Prüfung bei allen cookie-authentifizierten Mutationen. Logout widerruft nur die aktuelle Sitzung.

2 · Wireframes

Digital erstellte Low-Fidelity-Skizzen der sieben implementierten Seiten (Struktur/Anordnung, keine finale Gestaltung).



3 · Baumstruktur (Komponentenbaum)

```
RootLayout [Server]
├── Home [Server] → Links zu Registrierung/Anmeldung
├── RegisterPage [Server] → Card → RegisterForm [Client]
├── LoginPage [Server] → Card → LoginForm [Client]
├── geschützte Bereiche [Server; AppNavigation + Logout auf jeder Seite]
│   ├── AppNavigation [serverkompatibel] → Links, Guthaben-Chip, LogoutControl [Client]
│   ├── LobbyPage → Card (Name/Credits) + Links zu Dice/Roulette/Rangliste
│   ├── DicePage → Card → DiceGame [Client]
│   ├── RoulettePage → Card → RouletteGame [Client]
│   └── LeaderboardPage → Card → Ranglistentabelle
```

Fünf Client Components (RegisterForm, LoginForm, LogoutControl, DiceGame, RouletteGame) — jede mit konkretem Interaktionsgrund (kontrollierter State, Pending-Feedback). Button, Input, Card, AppNavigation bleiben serverkompatibel. Alle Komponenten nutzen typisierte Props (ein Props-Objekt je Komponente); Formularfelder binden Eingaben per kontrolliertem React-State (value / onChange, echtes Two-Way-Binding der UI) — Domänenendaten wie Ergebnis/Guthaben fließen dagegen bewusst nur serverseitig → clientseitig. Layout: CSS Grid für Zentrierung/mehrspaltige Anordnung (Formulare, Dice-/Roulette-Steuerung, Lobby-Linkraster), Flexbox für Navigation, Aktionsreihen und responsives Umbrechen.

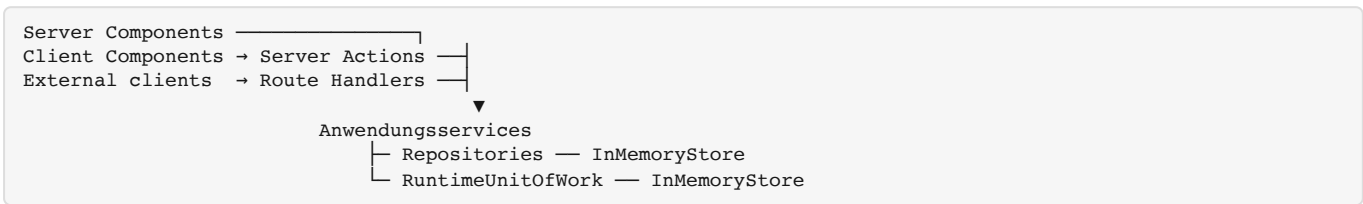
4 · Datenmodell

Objekt	Inhalt
Account	ID, Anzeigenname, normalisierte E-Mail, Passwort-Hash, materialisierter Creditstand, Erstellungszeitpunkt.
CreditTransaction	ID, Konto-ID, Runden-ID (oder null), vorzeichenbehafteter Betrag, Grund (STARTING_CREDIT / DICE_ROUND / ROULETTE_ROUND), resultierender Stand.
Session	ID, Konto-ID, SHA-256-Hash des Tokens, Erstellungs-/Ablaufzeitpunkt.
GameRound	ID, Konto-ID, Transaktions-ID, requestId, Spiel (DICE / ROULETTE), Einsatz, Spielinput, Ergebnis, Gewinnstatus, Nettoänderung, Endstand.

```
Account 1 — 0..* Session
Account 1 — 1..* CreditTransaction
Account 1 — 0..* GameRound
GameRound 1 — 1 CreditTransaction
```

IDs sind serverseitig erzeugte UUIDs; Zeitpunkte UTC-ISO. Passwort-Hashes und Sitzungstokens verlassen den Server nie als Client-DTO. Relationales Modell (statt Dokumentenspeicher) wäre die naheliegende Produktionswahl — sichert Eindeutigkeit, Beziehungen und atomare Mehrfach-Datensatz-Änderungen über Constraints/Transaktionen.

5 · Architektur



Kleine Schichtenarchitektur (Layered): Seiten/Actions/Route-Handler → Services → Repositories/Store. Passt zu den synchronen Registrierungs-, Auth- und Spielabläufen und lässt sich mit isolierten Services testen. Event-Driven wäre für diesen Umfang unnötige Zustellungs-/Konsistenzkomplexität; direkter gemeinsamer Datenzugriff (Data-Centered) würde die serverseitigen Fachregeln verwischen. Fachregeln liegen ausschließlich in Domain/Services; Store-Sammlungen bleiben privat; server-only-Module sind markiert; Dice und Roulette bleiben unabhängig lesbar und teilen nur Infrastruktur (Credit-Policy, atomare Schreibgrenze, Fehlercodes), keine generische Spiel-Engine.

6 · Schnittstellenbeschreibung

Server Actions bedienen die eigene Oberfläche (Registrierung, Login, Logout, Dice, Roulette); dieselben Anwendungsservices werden zusätzlich über eine ressourcenorientierte REST-API für externe/versionierte Zugriffe bereitgestellt (nicht streng zustandslos, da serverseitige Sitzungen).

Methode & Ressource	Autorität	Erfolg
GET /api/v1/users/me	Geprüfte Sitzung	200, sichere Konto-/Creditdarstellung
GET /api/v1/leaderboard	Geprüfte Sitzung, nur öffentliche Daten	200, Rang/Name/Credits
GET /api/v1/game-rounds	Besitzer serverseitig bestimmt	200, eigene Runden
POST /api/v1/game-rounds	Server bestimmt Ergebnis/Änderung	201 neu / 200 idempotent wiedergegeben
DELETE /api/v1/sessions/current	Geprüfte aktuelle Sitzung	204, kein Body

Fehler als application/problem+json: 401 (Auth fehlt), 403 (unsichere Mutation), 404, 409 (Request-ID-Konflikt), 422 (Validierung), 429 (Rate-Limit, mit Retry-After), 500 (neutral). Zod validiert jede Grenze strikt (unbekannte Felder abgelehnt).

7 · DEV-Guidelines

- App Router & Server Components als Standard; Client-Grenzen nur bei echtem Interaktionsgrund.
- TypeScript strict, kein `any`; unbekannte Grenzwerte als `unknown` + Validierung.
- camelCase für Variablen/Funktionen, PascalCase für Komponenten/Typen; einheitliches deutsches Domänenvokabular.
- Stabile Domänenzustände als Enums/ `as const` -Unions statt freier Strings.
- Jede visuelle Komponente hat ein eigenes `*.module.css` (keine Inline-Styles); UI-Atoms wie `Button` modellieren Varianten (`primary` / `secondary` / `outlined`) typisiert statt über freie Strings.
- Jede externe Grenze validiert unbekannte Daten serverseitig (nie nur clientseitig).
- Keine hardcodierten Secrets — ausschließlich über Umgebungsvariablen.
- Store-Sammlungen bleiben privat; Repositories/atomare Operationen sind auf konkrete Anwendungsfälle begrenzt.
- Neue Funktionen erhalten positive, negative und Grenzfalltests; keine Testendpunkte im Produktivcode.
- Business-Konstanten (Startguthaben, Sitzungsdauer, Rate-Limits) liegen zentral in typisierten Konstanten/Policies statt als Magic Numbers in Komponenten.

8 · Test Cases (Auswahl)

Testfall	Ebene	Erwartetes Ergebnis
Gültige Registrierung	Integration	Genau ein Konto + eine Starttransaktion (100 Credits).
Doppelte/ungültige E-Mail	Integration	Normalisierte Duplikate und ungültige Eingaben ohne Schreibvorgang abgelehnt.
Gültige/ungültige Anmeldung	Integration	Korrekte Zugangsdaten erzeugen eine 8h-Sitzung; falsche/unbekannte liefern dieselbe neutrale Meldung.
Geschützter Zugriff ohne Sitzung	Unit/E2E	Zugriff und Mutation werden serverseitig abgelehnt, Proxy-Guard ersetzt keine Prüfung.
Dice-/Roulette-Gewinn & Verlust	Integration, Komponente, E2E	Server bestimmt Ergebnis; genau eine <code>GameRound</code> + passende <code>CreditTransaction</code> .
Unbezahlbarer/ungültiger Einsatz	Integration	Abgelehnt ohne Zufallszug und ohne Mutation.
Idempotente Wiederholung (<code>requestId</code>)	Integration	Gleiches Ergebnis mit <code>replayed: true</code> , kein zweiter Zug/keine zweite Mutation.
Atomarer Schreibfehler	Integration	Konto, Saldo, Runden, Idempotenzbindung bleiben unverändert.
Leaderboard-Sortierung/Ties	Integration/E2E	Absteigend nach Credits, faire Rangvergabe, keine Zugangsdaten sichtbar.
Logout-Isolation	Integration	Nur aktuelle Sitzung entfernt; zweite Sitzung/Konto/Credits bleiben erhalten.
Rate-Limit-Überschreitung	Unit/Integration	429 mit <code>Retry-After</code> bzw. sichere Fehlermeldung, geschützte Aktion nicht ausgeführt.

Vollständige Suite: **165 automatisierte Tests** (Vitest, Unit/Komponente/Integration) + **9 Playwright-E2E-Abläufe** gegen einen Produktionsbuild — vollständig einsehbar unter `tests/` im Repository. Registrierung, Authentifizierung, Dice und Roulette wurden jeweils per RED-Commit vor der Implementierung festgelegt (TDD); RED-Tests kompilierten und scheiterten an fachlichen Assertions, nicht an fehlenden Importen.

9 · Konzept: Rendering (SSR/CSR)

`/`, `/register`, `/login` sind statisch erzeugbare Server-Component-Seiten; Formulare hydratisieren client-seitig. `/lobby`, `/dice`, `/roulette`, `/leaderboard` werden dynamisch gerendert (Sitzungsprüfung). Der Proxy prüft nur grob das Cookie; jede Seite/Mutation validiert die Sitzung serverseitig erneut — die Anwendung ist also keine reine CSR-Anwendung, sondern nutzt Server-Rendering, Hydration und gezielte Client-Inseln.

10 · Mögliches verteiltes System

Browser → Reverse Proxy → mehrere Next.js-Instanzen
└─ relationale Datenbank
└─ gemeinsamer Sitzungsspeicher

Die aktuelle Anwendung ist bewusst kein verteiltes Backend (Zielumfang: Einzelprozess). Für Mehrinstanzbetrieb wäre eine relationale Datenbank (Constraints/Transaktionen für Konten, Credits, Runden) und ein gemeinsamer Sitzungsspeicher nötig, damit Login/Logout auf allen Instanzen konsistent sind. Ein Load Balancer allein löst getrennte In-Memory-Zustände nicht. Schlägt ein gemeinsamer Speicher vor dem Commit fehl, darf keine Instanz einen lokalen Ersatzstand erzeugen; die Mutation schlägt sicher fehl statt inkonsistent zu schreiben. Wiederholungen sind nur für idempotente Lesezugriffe oder gegen Duplikate geschützte Mutationen zulässig.

KI-Einsatz (Zusammenfassung)

KI wurde als Werkzeug für Strukturierung, Entwürfe, Implementierungsunterstützung, Tests und redaktionelle Überarbeitung eingesetzt — nach dem Muster PRD → RED-Test → GREEN-Implementierung → Review, mit projektinternen Skills (`development`, `security`, `testing`, `quality-review`) und einem separaten read-only QA-Agenten für unabhängige Audits. Der Studierende gab Anforderungen und Umsetzungsweise vor, prüfte jeden Diff, bewertete Test-/Buildnachweise und traf alle Korrektur- und Freigabeentscheidungen. Vollständiges, nach Phasen gegliedertes Protokoll: `docs/ai-usage.md` im Repository.

Formale Rahmenbedingungen: Einzelarbeit · Next.js als Framework · Repository- und PDF-Name = Matrikelnummer 2050206 · private GitHub-Repository mit Contributor `wdski25b` · ≥10 inhaltlich nachvollziehbare Commits · Abgabe über GitHub und Moodle · Frist 16.08.2026, 23:59 Uhr.